

Dynamic Obstacles

Making the world move underneath your agents.

~15 minutes. Assumes Tutorial 03

IN SHORT

Add an `LCM Nav3D Dynamic Obstacle` component to any actor that moves. It restamps the octree as the actor moves and agents reroute around it: closing doors, rising platforms, patrolling hazards, destructible cover.

Static vs dynamic

When the navigation volume is built, all static geometry is baked in. That bake is what makes queries fast, but it's a snapshot.

- **Never moves?** Do nothing. It's already in the octree.
- **Moves?** It needs a **Dynamic Obstacle** component, or the octree will keep believing the space is empty and agents will fly straight through it.

NOTE

The common mistake: moving a static-mesh actor at runtime *without* the component. Nothing errors: agents ignore it, because as far as the navigation data is concerned it never moved.

Step 1: Make a moving obstacle

1. Create a Blueprint Actor, `BP_MovingWall`.
2. Add a **Static Mesh** component: a Cube, scaled to something that genuinely blocks a route (say $400 \times 100 \times 400$).
3. Make sure the mesh **has collision enabled**. The voxelizer finds obstacles via collision, so a mesh set to *No Collision* is invisible to navigation.

Step 2: Add the Dynamic Obstacle component

1. In `BP_MovingWall`, Add Component → search "LCM Nav3D Dynamic Obstacle".
2. That's the whole setup. It auto-registers with the navigation manager at `BeginPlay` and tracks its owner from then on.

The settings

Field	Default	What it does
Update Distance Threshold	50 cm	How far the actor must move before the octree is restamped. Larger = cheaper, but navigation lags reality.
Tracked Components	(empty)	Empty means the whole actor . Pick specific components to track only part of it.

`Tracked Components` is worth knowing about. On an actor where only one part obstructs (a gate inside a decorative frame, a swinging arm on a fixed base), pick just that component. Only its movement restamps the octree, and only its bounds block agents.

You can also manage this at runtime from Blueprint with `Add Tracked Component` and `Remove Tracked Component`.

Step 3: Move it

Give it motion. The simplest version, in the Event Graph:

```
Event Tick → SetActorLocation( StartLocation + (0, sin(GameTime) * 700, 0) )
```

Or use a Timeline, a matinee/sequencer track, physics; it doesn't matter. The component watches the transform, however it changes.

Step 4: See it work

1. Place `BP_MovingWall` so it periodically **seals the only route** between your flyer and its goal.
2. Press **Play**.

The agent should path around the wall, and when the wall closes the route, re-plan and take a different way.

To watch it happen: tick `Draw Debug Volumes` on the navigation manager. You'll see the voxels under the wall flip to blocked as it moves. That's the restamp.

How agents notice

Agents re-plan automatically. The relevant setting is on the **navigation manager**:

Field	Default	Meaning
4. Performance → Finite Obstacle Update Interval	0.1s	How often obstacle changes are folded into the finite-world octree

Lower = more reactive, more CPU. **0.1** is a good balance. For fast-moving hazards where agents must react instantly, try **0.05**.

Ready-made examples

Six demo maps show this working. Open any of them and read the setup:

Map	Shows
Demo/BehaviourTree/DynObs_Finite_BT	Three obstacles sealing routes in a finite world
Demo/BehaviourTree/DynObs_Infinite_BT	Five gates in a streaming infinite world
Demo/StateTree/DynObs_Finite_ST · DynObs_Infinite_ST	The StateTree equivalents
Demo/MassEntity/DynObs_Mass	Dynamic obstacles against a Mass crowd
Demo/MassEntity/VolumetricCrowd_LiveDemo	A gate sealing across a crowd flow

The obstacles in these maps are driven by the **LCM Nav3D Obstacle Mover** component (below), so you can see a complete working rig rather than assembling one.

The bundled Obstacle Mover

The plugin ships a small helper so the demo maps move without Blueprint wiring, and it's useful in your own prototypes.

LCM Nav3D Obstacle Mover ping-pongs its owner between its start pose and an offset.

Field	Default	Meaning
Seal Offset	(400, 0, 0)	Vector from the open pose to the sealed pose
Period	4.0 s	Time for a full open → sealed → open cycle
Phase	0.0 s	Offset so several obstacles don't move in lockstep

Pair it with a Dynamic Obstacle component on the same actor and you have a moving obstacle with zero scripting. It moves the actor kinematically, which is exactly what the obstacle tracker expects.

NOTE

There's a companion `LCM Nav3D Agent Driver` used by the demo maps to loop a pawn between two tagged points (`LcmGoalA` / `LcmGoalB`) using the real async navigation stack, handy for testing a level without authoring a full BT.

Performance notes

Dynamic obstacles are not free: each restamp re-voxelizes a region.

Keep it cheap:

- Prefer **fewer, larger** dynamic obstacles over many small ones.
- Raise `Update Distance Threshold` for things that move constantly but don't need precision.
- Use `Tracked Components` so only the genuinely blocking part is considered.
- If something moves but can never block a route (debris, VFX props, decoration), **don't give it the component at all**.

Don't put a Dynamic Obstacle component on your navigating agents. Agents avoid each other through the avoidance system ([Tutorial 03 Step 7](#)); making them obstacles too means each one is constantly re-voxelizing the world around itself.

Troubleshooting

Symptom	Cause
Agents fly straight through the obstacle	No Dynamic Obstacle component, or the mesh has collision disabled
Reaction is sluggish	<code>Update Distance Threshold</code> too high, or <code>Finite Obstacle Update Interval</code> too long
Frame rate drops when things move	Too many dynamic obstacles, or they're too finely voxelized: raise <code>Min Voxel Size</code> or reduce obstacle count
Agent gets trapped when a route seals	Expected if there's genuinely no other route. Give the level a second path, or handle the failure in your BT/StateTree

More: [Troubleshooting](#).

Where to go next

You now have the whole core loop: a navigation volume, agents that fly through it, and a world that can change underneath them. From here, pick what your project needs:

If you need...	Go to
Thousands of agents	Tutorial 06: Mass Entity
AI that picks tactical positions	Tutorial 07: Tactical EQS
Sight and awareness	Tutorial 08: Perception
Ground and flying AI together	Tutorial 09: Hybrid with Recast
Custom traversal or cost zones	Tutorial 10: Nav Links & Area Modifiers
To inspect it live in-game	Tutorial 11: Gameplay Debugger
To tune every setting	Settings Reference

Chapter 5 of the LCM Nav3D User Manual.

The complete manual is [LCM_Nav3D_Manual.pdf](#), in this folder alongside every other document in the set. Start from [README.pdf](#).

LCM Nav3D: True Volumetric 3D Navigation for Unreal Engine. Version 2.05 · Unreal Engine 5.8.

Copyright © 2026 L.Charitha Madhushanka. All Rights Reserved. Proprietary and confidential. Licensed, not sold; use is governed by the licence that accompanies this software. Unreal Engine is a trademark of Epic Games, Inc. All trademarks are the property of their respective owners.